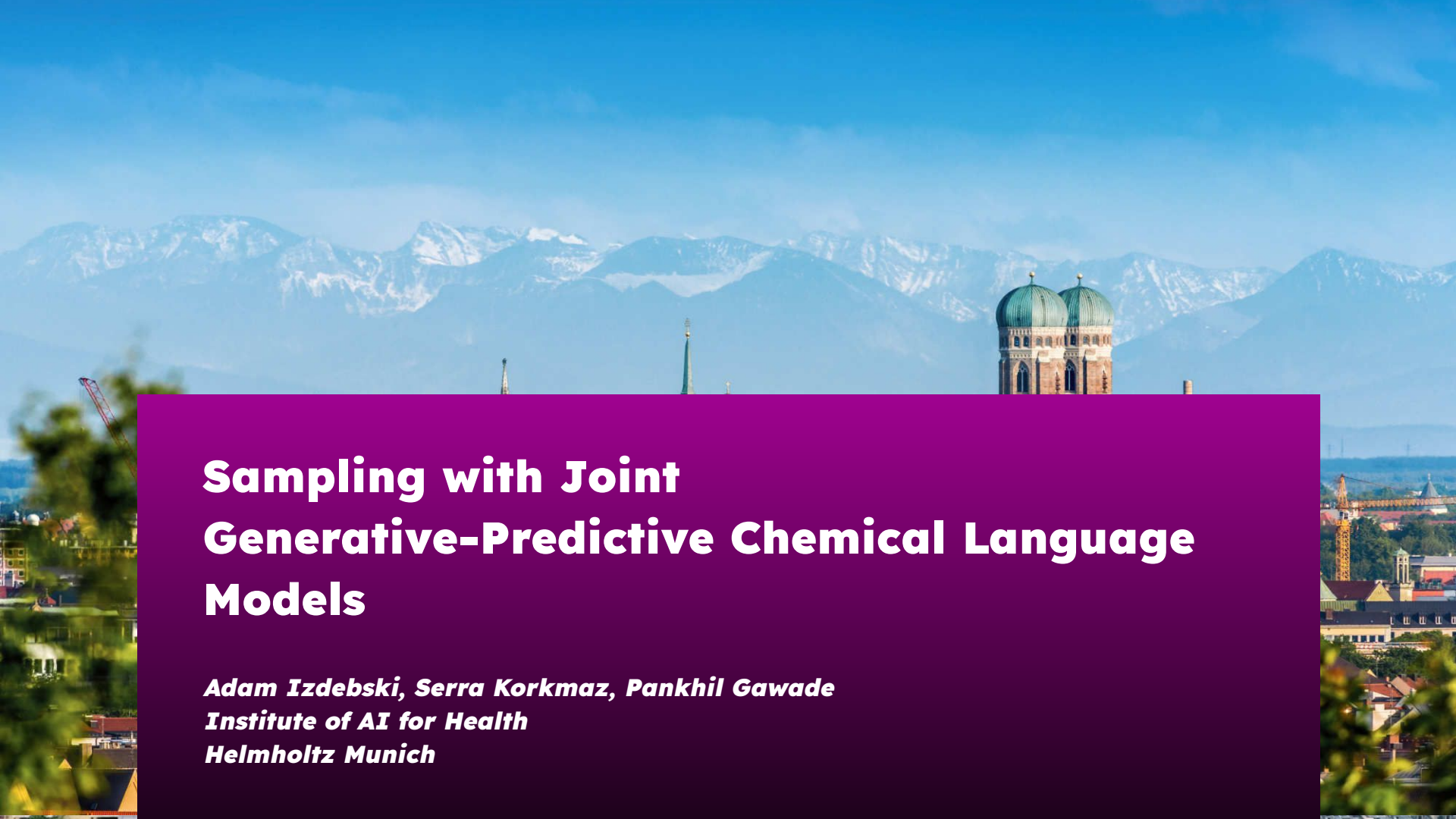


Sampling with Joint Generative-Predictive Chemical Language Models

Adam Izdebski, Serra Korkmaz, Pankhil Gawade
Institute of AI for Health
Helmholtz Munich

Agenda

1. Adam Izdebski: Hyformer (30 minutes)
2. Serra Korkmaz: Joint Self-Improvement (30 minutes)
3. Pankhil Gawawe: AMPformer (30 minutes)

Hyformer: Synergistic Benefits of Joint Molecule Generation and Property Prediction

Izdebski, A., Olszewski, J., Gawade, P., Koras, K., Korkmaz, S., Rauscher, V., Tomczak, J. M., & Szczurek, E. (2026). Synergistic Benefits of Joint Molecule Generation and Property Prediction. Transactions on Machine Learning Research (TMLR), ICLR 2026.

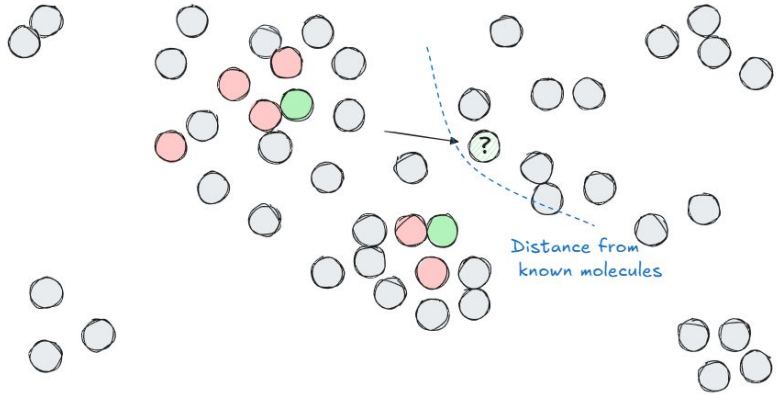
HELMHOLTZ MUNICH

De novo Drug design

Motivation. De novo drug design requires balancing generation of new molecules with robust property prediction, to prioritize active molecules for *in-vitro* validation.

De novo Drug design

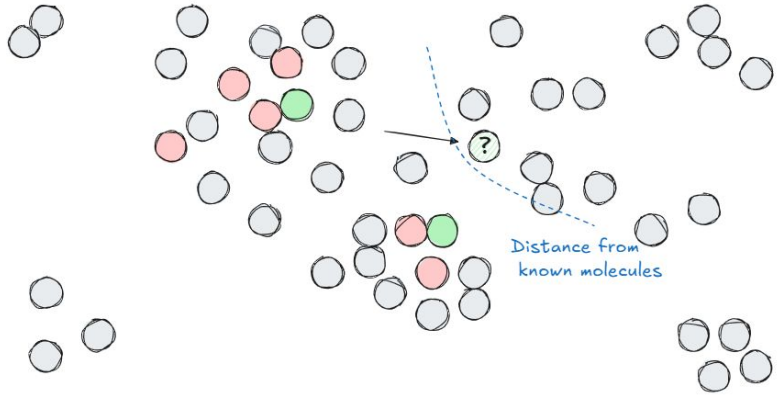
Motivation. De novo drug design requires balancing generation of new molecules with robust property prediction, to prioritize active molecules for *in-vitro* validation.



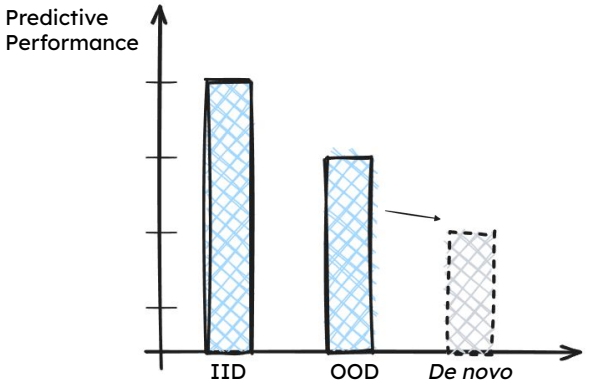
In drug design, we predict the activity of molecules that are “far” from known molecules (OOD) and/or generated *de novo* (Synthetic).

De novo Drug design

Motivation. De novo drug design requires balancing generation of new molecules with robust property prediction, to prioritize active molecules for *in-vitro* validation.



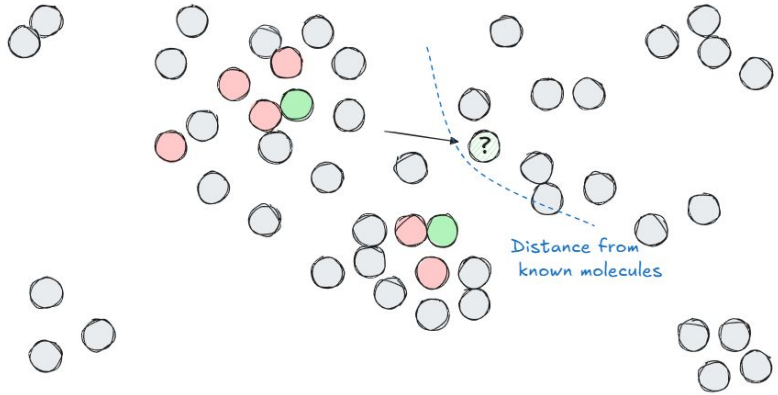
In drug design, we predict the activity of molecules that are “far” from known molecules (OOD) and/or generated *de novo* (Synthetic).



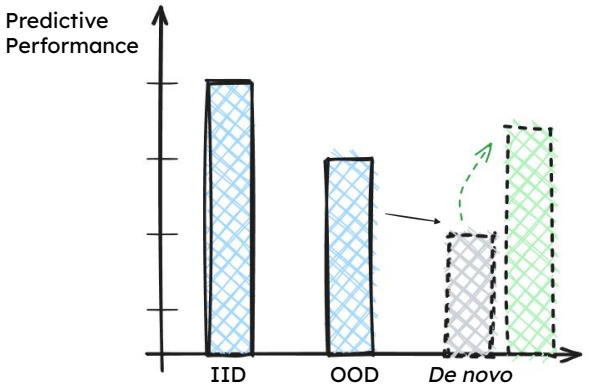
The predictive robustness of property predictors can drop for synthetically generated data.

De novo Drug design

Motivation. De novo drug design requires balancing generation of new molecules with robust property prediction, to prioritize active molecules for *in-vitro* validation.



In drug design, we predict the activity of molecules that are “far” from known molecules (OOD) and/or generated *de novo* (Synthetic).



The predictive robustness of property predictors can drop for synthetically generated data.

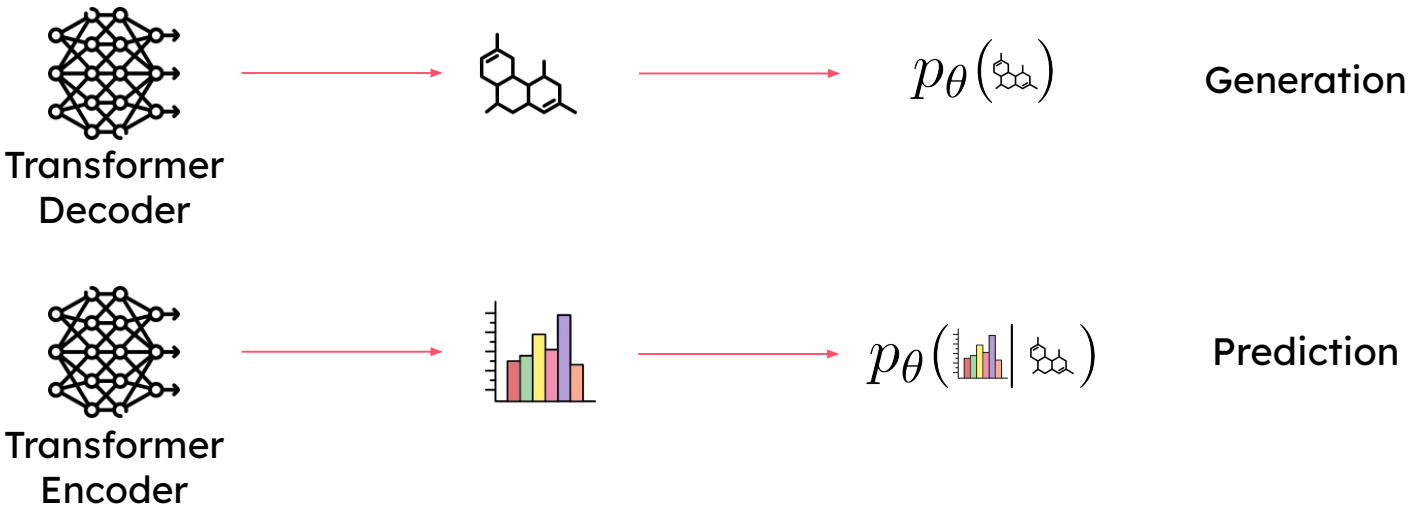
Joint Modeling

Joint Modeling

Idea. Generation and prediction can be unified within a single architecture and a single set of model weights.

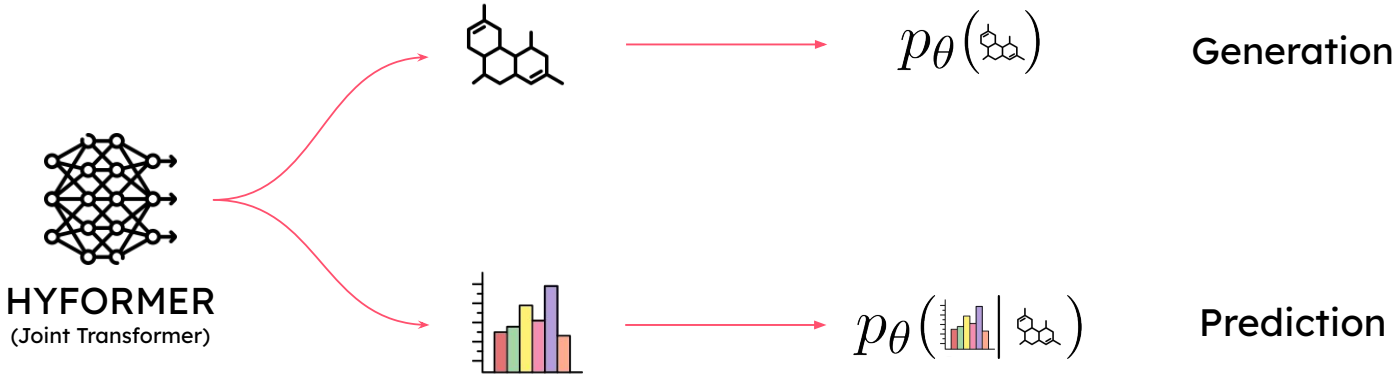
Joint Modeling

Idea. Generation and prediction can be unified within a single architecture and a single set of model weights.



Joint Modeling

Idea. Generation and prediction can be unified within a single architecture and a single set of model weights.



$$p_{\theta,\phi}(\mathbf{x}, y) = p_\phi(y \mid \mathbf{x}) p_\theta(\mathbf{x})$$

Synergistic Benefits: conditional sampling

Contribution. Show synergistic benefits of joint modeling for conditional sampling.

Synergistic Benefits: conditional sampling

Contribution. Show synergistic benefits of joint modeling for conditional sampling.

Baselines.

1. Append the condition to sequence and sample directly from p(x|y) (MolGPT, GraphGPT)
2. Train a classifier, use it to rank N-many samples and select the best one (Best-of-N)

Synergistic Benefits: conditional sampling

Contribution. Show synergistic benefits of joint modeling for conditional sampling.

Baselines.

1. Append the condition to sequence and sample directly from p(x|y) (MolGPT, GraphGPT)
2. Train a classifier, use it to rank N-many samples and select the best one (Best-of-N)

Hyformer:

```python
def sample(target: float, model: JointModel, N: int, epsilon: float):

    # unconditionally sample N-many examples
    samples = model.generate(num_samples=N)

    # predict target properties
    targets = model.predict(samples)

    # accept, if within an \epsilon range from `target` value
    accepted_samples = samples[torch.abs(targets - target) < epsilon]

    return accepted_samples
```

Synergistic Benefits: conditional sampling

Contribution. Show synergistic benefits of joint modeling for conditional sampling.

Baselines.

1. Append the condition to sequence and sample directly from p(x|y) (MolGPT, GraphGPT)
2. Train a classifier, use it to rank N-many samples and select the best one (Best-of-N)

Hyformer:

```python
def sample(target: float, model: JointModel, N: int, epsilon: float):

    # unconditionally sample N-many examples
    samples = model.generate(num_samples=N)

    # predict target properties
    targets = model.predict(samples)

    # accept, if within an \epsilon range from `target` value
    accepted_samples = samples[torch.abs(targets - target) < epsilon]

    return accepted_samples
```

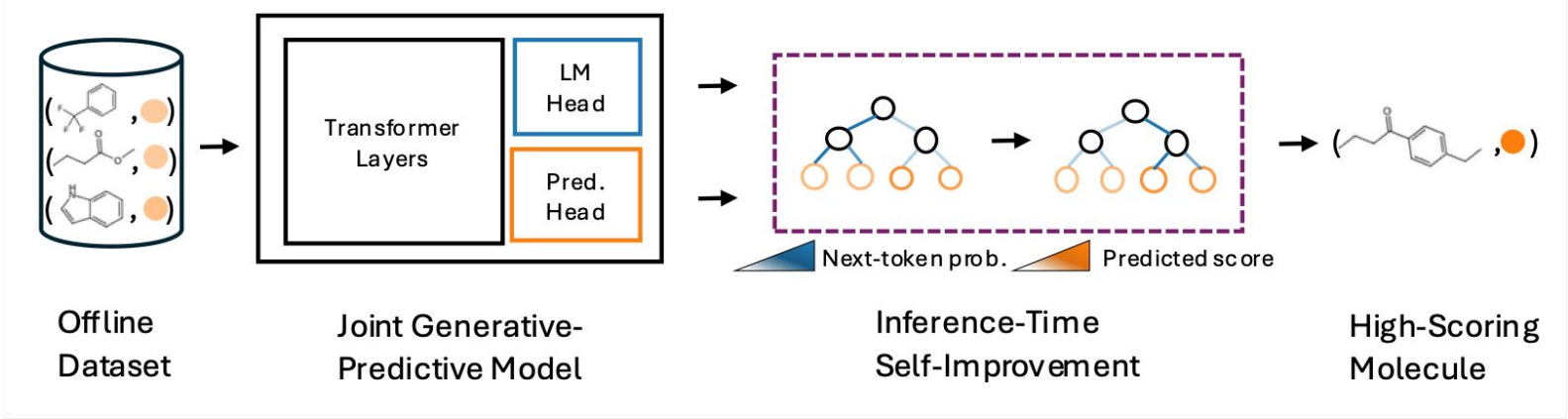
Table 1: Conditional generative performance on GuacaMol dataset. Best model is marked **bold**.

Model	Joint	Metric	QED	SA	logP	Avg.
MolGPT	✗	MAD ↓	0.087	0.019	0.276	0.127
		SD ↓	0.074	0.017	0.262	0.118
		Validity ↑	0.985	0.986	0.982	0.984
GraphGPT	✗	MAD ↓	0.039	0.011	0.158	0.069
		SD ↓	0.082	0.047	0.653	0.261
		Validity ↑	**0.998**	**0.997**	0.992	**0.995**
Hyformer	✗	MAD ↓	0.029	0.014	0.154	0.066
		SD ↓	0.041	0.018	0.199	0.086
		Validity ↑	0.991	0.977	0.991	0.986
	✓	MAD ↓	**0.008**	**0.005**	**0.026**	**0.013**
		SD ↓	**0.013**	**0.008**	**0.033**	**0.018**
		Validity ↑	0.995	0.983	**0.995**	0.991

Joint Self-Improvement

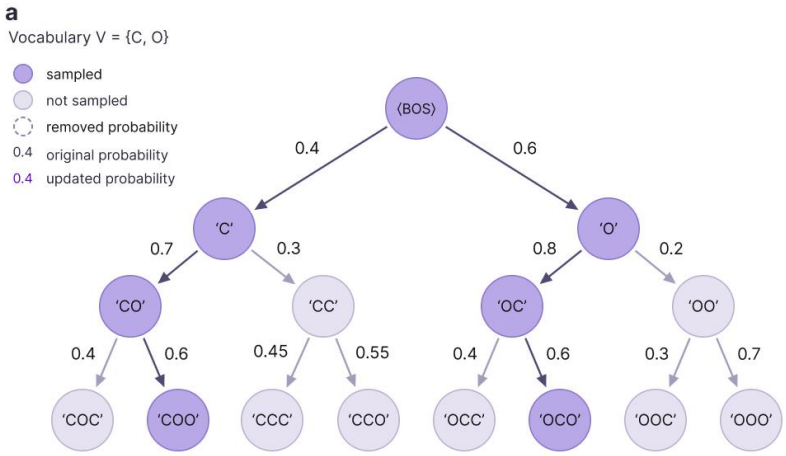
Korkmaz, S., Izdebski, A., Pirnay, J., Møller-Larsen, R., Kmicikiewicz, M., Gawade, P., Pavlov, M., Grimm, D. G., & Szczurek, E. (2026). Sample efficient generative molecular optimization with joint self-improvement. In Advances in Neural Information Processing Systems

JSI: Joint Self-Improvement



Joint generative-predictive model is fine-tuned on an offline dataset, a labeled dataset of molecules and their property scores. At inference time, the model self-improves by iteratively biasing next-token probabilities (blue) using predicted property scores (orange), steering generation towards high-scoring molecules. In the online setting, the predictive head is replaced by an oracle, and the model is iteratively fine-tuned on the accumulating online dataset.

Inference-Time Self-Improvement



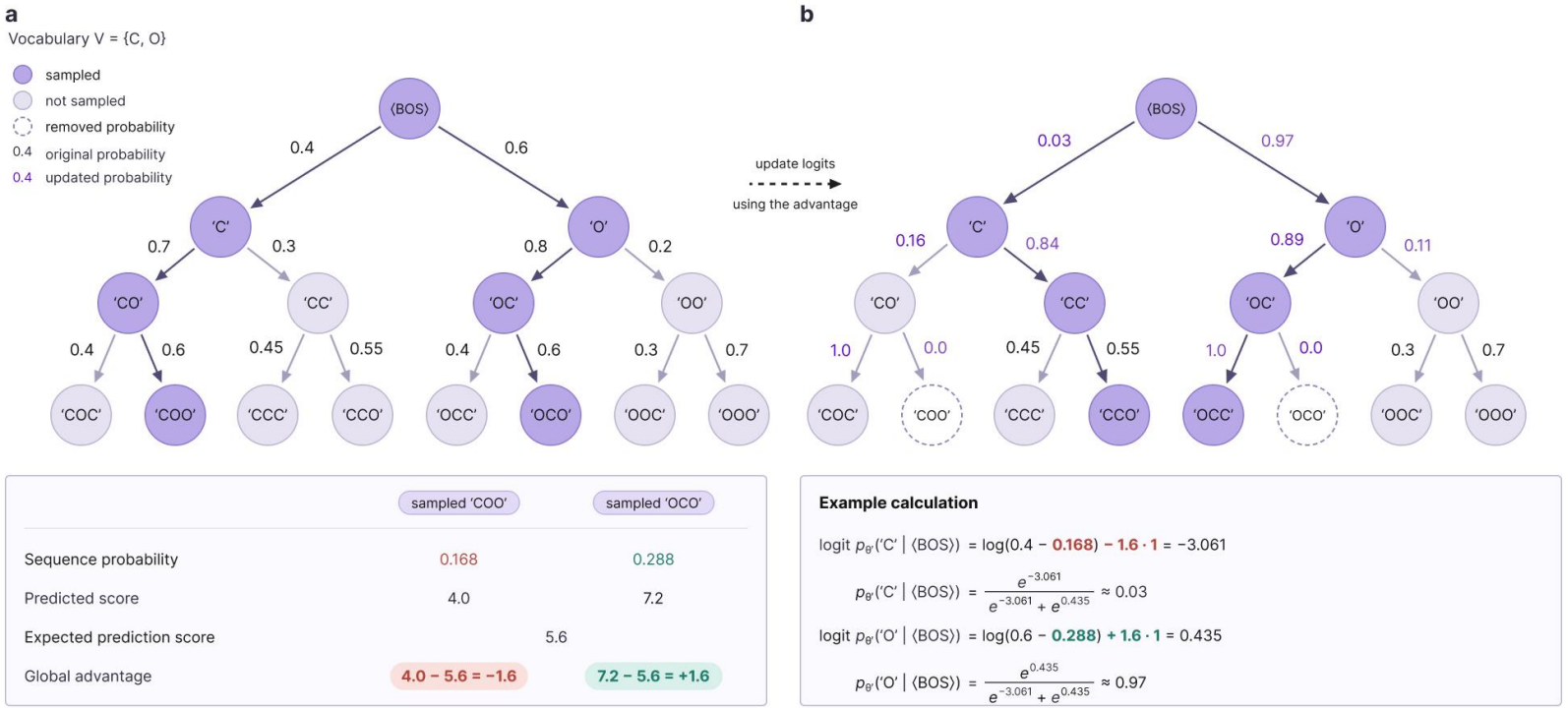
	sampled 'COO'	sampled 'OCO'
Sequence probability	0.168	0.288
Predicted score	4.0	7.2
Expected prediction score	5.6	
Global advantage	4.0 − 5.6 = −1.6	7.2 − 5.6 = +1.6

⟨BOS⟩ beginning-of-sequence token · V vocabulary · pθ′ updated policy · Global advantage = predicted score − expected prediction score

$$\operatorname{logit} p_{\theta'}\left(x_t^{(k)} \mid \mathbf{x}_{1:t-1}^{(k)}\right) := \log\left(p_\theta(x_t^{(k)} \mid \mathbf{x}_{1:t-1}^{(k)}) - \sum_{\mathbf{x}\in S(\mathbf{x}_{1:t}^{(k)})} p_\theta(\mathbf{x} \mid \mathbf{x}_{1:t-1}^{(k)})\right) + \sigma \sum_{\mathbf{x}\in S(\mathbf{x}_{1:t}^{(k)})} \mathrm{Adv}_\theta(\mathbf{x})$$

HELMHOLTZ MUNICH

Inference-Time Self-Improvement



$$\operatorname{logit} p_{\theta'}\left(x_t^{(k)} \mid \mathbf{x}_{1:t-1}^{(k)}\right) := \log\left(p_\theta(x_t^{(k)} \mid \mathbf{x}_{1:t-1}^{(k)}) - \sum_{\mathbf{x}\in S(\mathbf{x}_{1:t}^{(k)})} p_\theta(\mathbf{x} \mid \mathbf{x}_{1:t-1}^{(k)})\right) + \sigma \sum_{\mathbf{x}\in S(\mathbf{x}_{1:t}^{(k)})} \mathrm{Adv}_\theta(\mathbf{x})$$

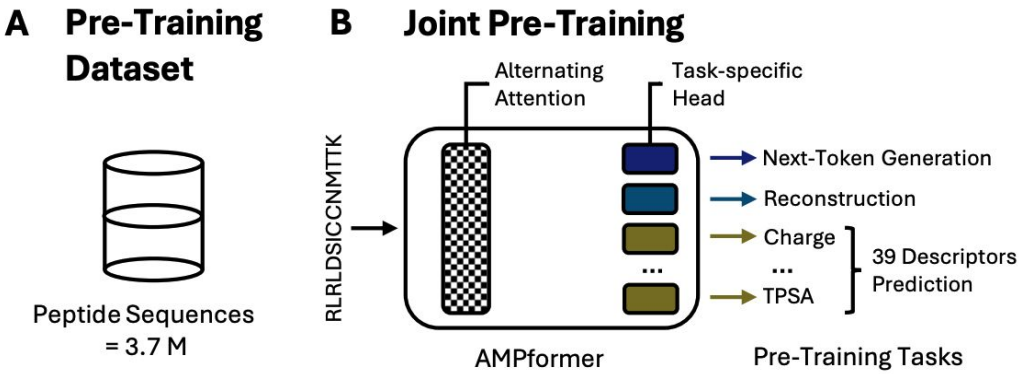
HELMHOLTZ MUNICH

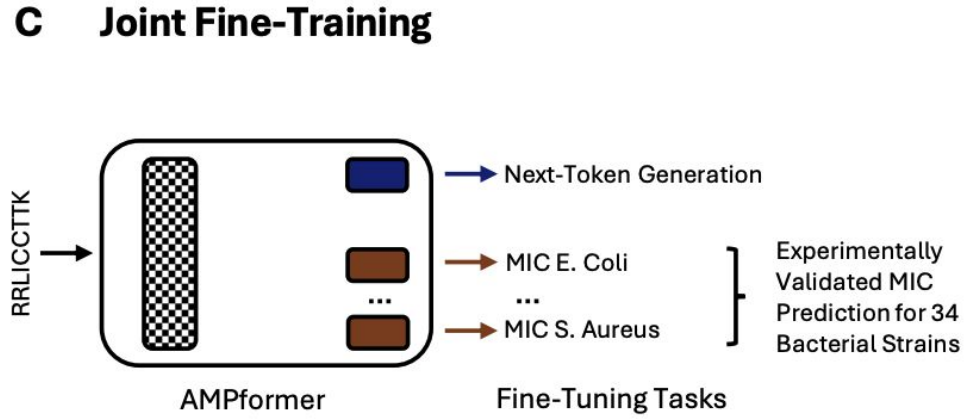
AMP Optimization

Table 5: Molecular optimization performance on offline AMP optimization with restricted proportion of hits. Best method is **bold**.

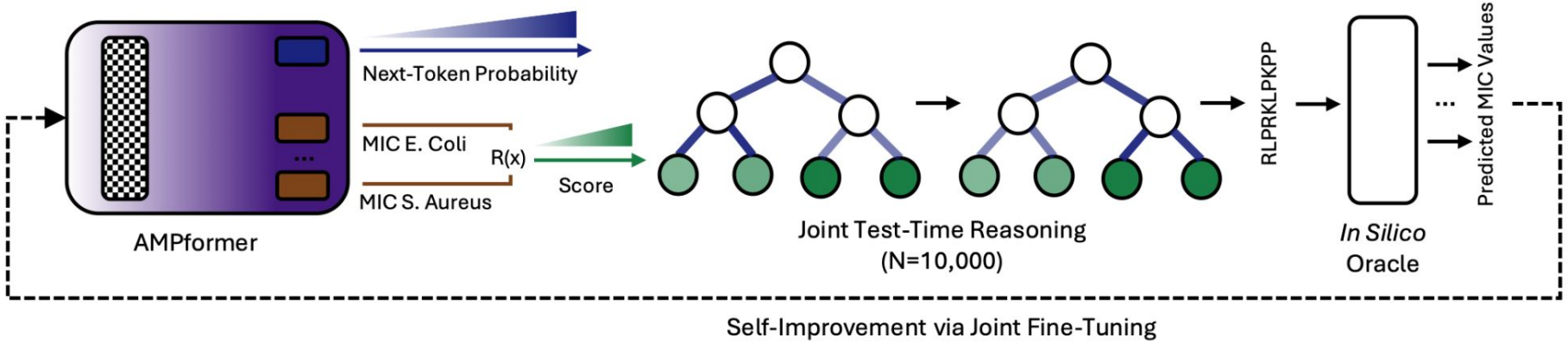
MODEL	PERPLEXITY	DIVERSITY (↑)	FITNESS (↑)	HYDRAMP$_{MIC}$ (↑)	AMPLIFY (↑)	AMPEPPY (↑)
1% HITS						
HYFORMER	16.84 (16.59, 17.09)	**0.83 (0.82, 0.83)**	0.11 (0.11, 0.12)	0.20 (0.18, 0.23)	0.76 (0.74, 0.78)	0.65 (0.64, 0.66)
JSI (OURS)	18.36 (17.94, 18.77)	0.69 (0.68, 0.70)	**0.16 (0.16, 0.17)**	**0.68 (0.65, 0.71)**	**0.98 (0.97, 0.98)**	**0.72 (0.72, 0.73)**
0% HITS						
HYFORMER	16.77 (16.51, 17.01)	**0.83 (0.83, 0.83)**	0.11 (0.10, 0.11)	0.17 (0.15, 0.19)	0.71 (0.69, 0.73)	0.62 (0.61, 0.63)
JSI (OURS)	18.34 (18.05, 18.66)	0.72 (0.72, 0.73)	**0.19 (0.18, 0.20)**	**0.66 (0.63, 0.69)**	**0.98 (0.97, 0.99)**	**0.71 (0.70, 0.72)**

AMPformer


A Pre-Training Dataset
Peptide Sequences = 3.7 M
B Joint Pre-Training
Alternating Attention
Task-specific Head
RLRLDSICCNMTTK
Next-Token Generation
Reconstruction
Charge
...
TPSA
39 Descriptors Prediction
AMPformer
Pre-Training Tasks


C Joint Fine-Training
RRLICCTTK
Next-Token Generation
MIC E. Coli
...
MIC S. Aureus
Experimentally Validated MIC Prediction for 34 Bacterial Strains
AMPformer
Fine-Tuning Tasks

Self-Improving Active Learning Loop with Joint Test-Time Reasoning



Control Knobs

→ Model sampling parameter (Temperature)

→ Rejection sampling

→ Fine-tuning

→ Oracle based steering [Take a Step and Reconsider https://arxiv.org/abs/2407.17206]